

RAG Multi-Format Test Document

This document intentionally contains multiple content modalities so it can be used to practice PDF ingestion, text extraction, table extraction, Markdown-aware processing, image handling, OCR, and multimodal RAG.

1. Normal Text Content

Retrieval-Augmented Generation (RAG) combines information retrieval with large language model generation. A typical pipeline loads documents, extracts content, chunks the content, creates embeddings, retrieves relevant passages, optionally reranks them, and supplies the selected context to an LLM.

A production RAG system must handle more than plain paragraphs. PDFs may contain tables, diagrams, forms, charts, scanned pages, mathematical expressions, and mixed layouts. These different structures require different extraction strategies.

2. Markdown Text and Data

```
## RAG Processing Stages
```

- Document ingestion
- Text and layout extraction
- Chunking and metadata enrichment
- Embedding generation
- Hybrid retrieval
- Reranking
- Context assembly
- LLM response generation

```
### Example Configuration
```

```
```yaml
retrieval:
 strategy: hybrid
 top_k: 10
 reranking: true
embedding:
 model: all-MiniLM-L6-v2
```
```

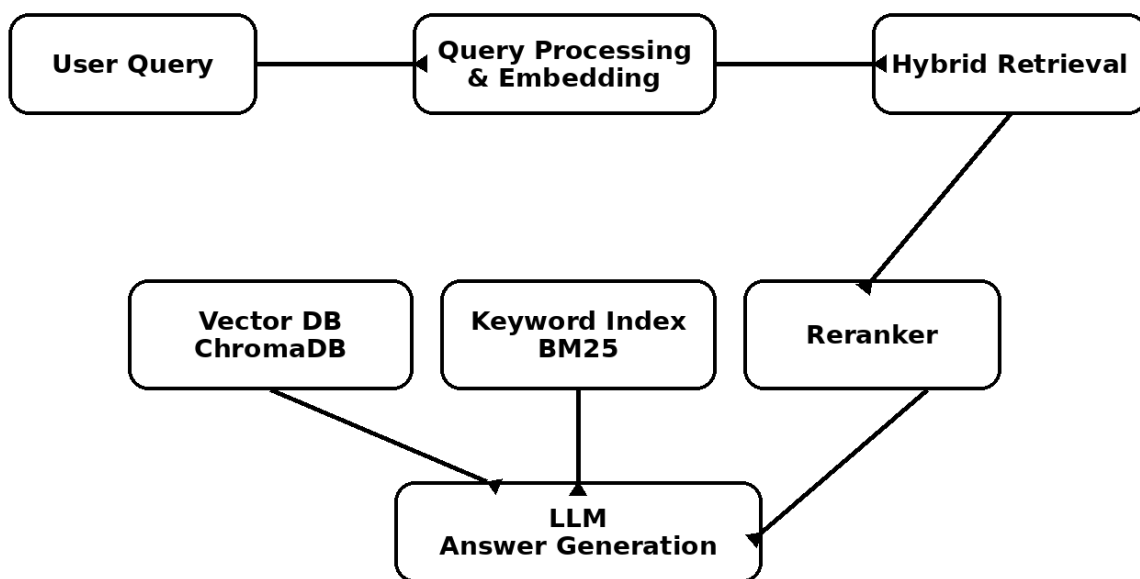
3. Structured Table Data

| Component | Purpose | Example Technology | Content Type |
|--------------|------------------------------|--------------------|--------------|
| Loader | Extract content from files | PyPDFLoader | Text / PDF |
| OCR | Read text from scanned pages | Tesseract | Image |
| Chunker | Split content for retrieval | Recursive Splitter | Text |
| Embedding | Convert content to vectors | MiniLM | Vector |
| Vector Store | Store and retrieve vectors | ChromaDB | Vector |
| Reranker | Improve retrieved relevance | Cross Encoder | Scores |

4. Architecture Diagram

The following image represents a simplified RAG architecture. In a RAG pipeline, the query can be transformed into an embedding and combined with keyword retrieval. Retrieved candidates may be reranked before being sent to the LLM.

Enterprise RAG Architecture



5. Scanned Form

The next page contains a rasterized form image rather than selectable PDF text. This is intentional: it allows OCR experiments using tools such as Tesseract and pytesseract.

DOCUMENT PROCESSING REQUEST FORM

Request ID: RAG-2026-0818-001

Document Type: Technical Architecture

Department: Data Engineering

Priority: High

Processing Mode: OCR + Layout Analysis

Requested By: Knowledge Engineering Team

Signature: _____ Date: 18-Aug-2026

OCR practice questions:

1. What is the Request ID?
2. What is the Document Type?
3. Which processing mode is requested?
4. What is the priority?

6. RAG Test Questions

Question 1: What technologies are listed in the table?

Question 2: What are the main stages of the RAG pipeline described in the Markdown section?

Question 3: What components appear in the architecture diagram?

Question 4: What is the Request ID in the scanned form?

Question 5: Which parts of this document would require OCR rather than normal PDF text extraction?

Question 6: Which content should ideally retain its structural representation instead of being flattened into plain text?